



ARIADNE: Adaptive UVM Management for Efficient GPU Memory Oversubscription

Hyunkyun Shin
Yonsei University
Seoul, Republic of Korea
hk.shin@yonsei.ac.kr

Seongtae Bang
DGIST
Daegu, Republic of Korea
st.bang@dgist.ac.kr

Hyungwon Park
DGIST
Daegu, Republic of Korea
hwpark@dgist.ac.kr

Daehoon Kim*
Yonsei University
Seoul, Republic of Korea
daehoonkim@yonsei.ac.kr

Abstract—Unified Virtual Memory (UVM) simplifies GPU programming and supports memory oversubscription, but suffers from severe performance degradation under high memory pressure due to page fault overhead and thrashing. Existing approaches such as prefetching, access counter-based migration, and dynamic Zero-copy offer limited benefits and often require hardware or compiler modifications, undermining UVM’s portability and ease of deployment. We present ARIADNE, a runtime UVM management framework that preserves UVM’s GPU memory abstraction while ensuring high and robust performance under memory oversubscription. ARIADNE is guided by three principles: (1) pipelined fault handling to hide migration latency, (2) Sharing Degree, a runtime metric that captures thread-level access locality without requiring hardware or compiler changes, to inform placement decisions, and (3) dynamic placement of memory regions between GPU memory and Zero-copy based on real-time access patterns. Implemented entirely within NVIDIA’s UVM driver, ARIADNE requires no recompilation or hardware modifications and applies transparently to any executable or closed-source GPU UVM applications. Our experimental results show that ARIADNE delivers average speedups of $1.9\times$, $5.0\times$, and $4.8\times$ over a state-of-the-art method at 130%, 175%, and 300% oversubscription, respectively, while effectively preventing thrashing and maintaining near-linear performance scaling.

Index Terms—GPU, Virtual memory, Memory management.

I. INTRODUCTION

While GPUs have become the de facto accelerator for workloads across various domains, including molecular dynamics, robotics, and artificial intelligence, their limited on-board memory remains a critical bottleneck for performance and usability [37], [43], [52], [56]. To address this, Unified Virtual Memory (UVM) [40] integrates the virtual address spaces of the CPU and GPU and transparently migrates the pages that are accessed by the GPU. By migrating only actively used pages to the GPU, UVM effectively prevents out-of-memory (OOM) errors and supports GPU memory oversubscription, the ability to use more memory than physically available on the GPU. Furthermore, this GPU memory abstraction significantly eases the programming burden by eliminating explicit management and application tuning, enhancing portability across different hardware configurations.

Despite its benefits, UVM often incurs significant performance degradation compared to explicit memory management, especially under memory oversubscription, where thrashing

becomes a critical issue [7], [34]. To mitigate these challenges, prior research has explored techniques broadly categorized into three groups: (1) prefetching, which proactively migrates data from host to device memory to reduce future page faults [16], [20], [28], [34], [36], [47], [54]; (2) access counter-based migration, which monitors page access frequency and migrates hot pages to the GPU based on fixed thresholds [17], [40]; and (3) alternative memory placement techniques such as Zero-copy, which place certain pages in host memory and access them remotely from the GPU without migration [7], [9], [11], [15], [19], [46], [58]. However, these solutions provide limited performance improvements and often compromise UVM’s key advantage, its abstraction of GPU physical memory and application portability across diverse hardware environments.

Prefetching can proactively migrate pages to reduce future page faults, but it remains ineffective at mitigating thrashing under high memory pressure. Access counter-based migration, which relies on fixed thresholds, performs reasonably well only under favorable conditions, but fails to adapt to rapidly changing access patterns in real-world workloads [17]. Dynamic Zero-copy techniques aim to improve page placement by pinning cold pages in host memory for remote GPU access, thereby avoiding unnecessary migrations. However, existing approaches require either hardware modifications or compiler-level instrumentation to track page hotness [7], [9], [15], [33]. Hardware-based solutions are often impractical in production environments, while compiler-assisted methods are unsuitable for applications distributed solely as executables or those with complex build dependencies.

To enhance UVM’s performance across various memory pressures without compromising its fundamental GPU memory abstractions, we propose ARIADNE¹, a runtime UVM framework, designed based on three design principles.

Design Principle #1. Reduce UVM fault-handling latency through pipelined execution (§IV-B). The UVM driver handles page faults when the GPU accesses non-resident pages, involving three core operations: *Populate* (GPU memory chunk allocation), *Eviction* (evicting chunks), and *Copy* (data

¹Ariadne, princess of Crete from Greek mythology, helped Theseus escape the Labyrinth by giving him a ball of *thread*. Analogously, ARIADNE leverages *thread*-level information to measure runtime page usefulness, achieving optimal page placement even in the UVM kernel module that operates with obscured runtime memory access information.

* Corresponding author.

transfer). However, fault-handling latency is too long to be effectively hidden by GPU context switching, reducing GPU utilization [30]. While prior works [30], [32] have parallelized *Eviction* with a sequential *Populate-Copy*, our observations reveal that *Populate* latency is nearly double that of the others, making it the primary bottleneck. ARIADNE addresses this by decoupling *Populate* from *Copy* and pipelining *Populate*, *Eviction*, and *Copy* operations across multiple VABlocks.

Design Principle #2. Sharing Degree: a runtime access-pattern metric leveraging thread-level information (§IV-A).

Because the UVM runtime driver has limited visibility into memory access behavior, prior works have modified hardware or compilers to collect access information. In contrast, by leveraging the GPU’s thread execution architecture, we identify a method to capture crucial thread-level information at runtime without hardware or compiler modifications. ARIADNE introduces the *Sharing Degree*, defined as the number of source unified Translation Lookaside Buffers (uTLBs) associated with recent faults for each memory range (VABlock). Since each uTLB can serve as a proxy for a thread group, the *Sharing Degree* effectively reflects the number of threads actively accessing a VABlock. A high *Sharing Degree* indicates dense access, while a low value suggests sparse access. To the best of our knowledge, this is the first runtime metric to exploit thread-level access information in UVM.

Design Principle #3. Managing GPU memory region placement between GPU memory and Zero-copy based on runtime memory access characteristics (§III-B, §V-D).

In UVM runtime, a virtual address space is managed at the 2 MB *Virtual Address Block* (VABlock) granularity, where each VABlock corresponds to a 2 MB GPU memory allocation unit called a chunk. Optimal performance under memory oversubscription requires orchestrating VABlock placement between GPU memory and Zero-copy based on access patterns. Given the contrasting access granularities of GPU migration (2 MB chunks) [40] and Zero-copy (128 B cache lines) [9], access sparsity is a natural criterion for guiding placement decisions. Since the *Sharing Degree* quantifies access sparsity for each VABlock, ARIADNE dynamically adjusts placement based on this metric. We introduce three runtime mechanisms: (1) a *memory demand monitor* based on the Working Chunk Set Size (WCSS), a UVM-specific metric that quantifies real-time memory demand to guide effective memory overcommitment, (2) a *Sharing-Degree-based eviction policy* that selects eviction victims based on fine-grained access locality, and (3) a *transient Zero-copy retention* strategy that keeps recently evicted, but still active, data in Zero-copy mode on the host memory to absorb transient reuse before migrating them back to GPU memory. These mechanisms keep high-utilization VABlocks in GPU memory longer while assigning sparsely accessed VABlocks to Zero-copy, ensuring optimal placement without thrashing or memory waste.

These three principles operate in concert: WCSS ensures precise memory demand tracking, *Sharing Degree* delivers resilient placement decisions, and pipelined fault handling conceals migration latency. Together, they enable ARIADNE

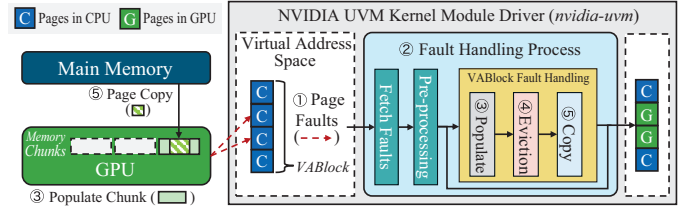


Fig. 1: UVM fault handling processes.

to maintain high performance under memory oversubscription while preserving UVM’s programmability and portability.

We evaluate ARIADNE on a real system using 10 diverse GPU benchmarks across varying memory oversubscription ratios. In our experiments, ARIADNE delivers average speedups of $1.9\times$, $5.0\times$, and $4.8\times$ over an existing SOTA approach at oversubscription levels of 130%, 175%, and 300%, respectively, while effectively preventing thrashing even at the highest level of 300%. Moreover, ARIADNE maintains near-linear performance scaling with oversubscription, increasing execution time by only $1.6\times$, $1.8\times$, and $2.3\times$ relative to the no-oversubscription case at these same levels. Because ARIADNE is implemented entirely within NVIDIA’s open-source UVM driver, requiring neither recompilation nor hardware changes, it can be applied transparently to any executable or closed-source UVM application. Consequently, ARIADNE advances the core mission of UVM, making GPGPU computing accessible under diverse user needs and memory constraints.

II. BACKGROUNDS

A. GPU Execution Architecture

To execute simultaneous operations across large datasets, Graphics Processing Units (GPUs) integrate numerous processing cores organized into clusters known as Streaming Multiprocessors (SMs) [39]. Each SM contains its own shared memory and L1 cache, and typically two adjacent SMs share a unified Translation Lookaside Buffer (uTLB) [41], [57]. CUDA programs, referred to as kernels, leverage this parallel hardware architecture by dividing computations into multiple threads. Threads are grouped into thread blocks (TBs), each of which is assigned to an SM for execution [32]. Threads and TBs are uniquely identified by thread IDs (tID) and block IDs (bID), respectively.

An important observation is that the execution model of GPU kernels, composed of multiple TBs and threads, closely resembles iterative operations in CPU programs. Just as CPU programs use loop variables (e.g., i) to select array indices during iterations, GPU kernels utilize tID and bID to determine the specific data accessed by each thread. Consequently, tID and bID play a crucial role in shaping memory access patterns of GPU applications [7], [29], [50], [51].

B. Unified Virtual Memory

Unified Virtual Memory (UVM) enables the CPU and GPU to share a single virtual address space and abstracts GPU memory management, allowing applications to transparently

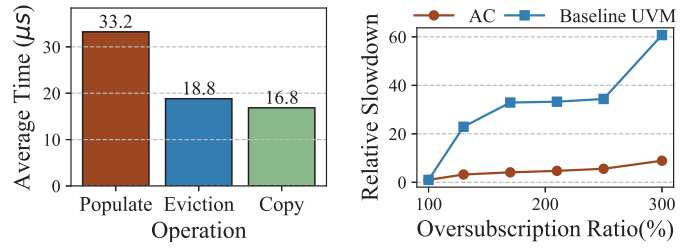
execute even on systems with limited GPU memory capacity without requiring code modifications [18]. Memory allocated via `cudaMallocManaged` is automatically managed by the UVM driver, removing the need for explicit data transfer operations such as `cudaMemcpy`. When a page managed by UVM (i.e., a UVM page) residing in host memory is accessed by the GPU, the UVM driver transparently migrates it into GPU memory.

To enhance efficiency, UVM manages memory at a granularity coarser than individual pages by grouping contiguous virtual address ranges into **VABlocks** and allocating GPU physical memory in larger units called **chunks** [40]. Typically, a VABlock and a chunk each correspond to a 2 MB region of virtual memory and GPU physical memory, respectively [3]. Chunks serve as the atomic units for physical memory allocation on the GPU and are organized through an eviction queue, which tracks chunk residency and facilitates selective eviction under memory oversubscription. Each VABlock maintains metadata, such as a page residency mask, to monitor the residency of individual pages within GPU memory, guiding runtime decisions regarding eviction, migration, and Zero-copy optimization.

By managing memory through VABlocks and chunks, UVM efficiently supports GPU memory oversubscription, allowing allocations that exceed the GPU’s physical memory capacity. Such capabilities are becoming increasingly critical due to the widespread adoption of GPUs in diverse applications and the growing disparity between GPUs’ substantial computational capabilities and their relatively limited memory capacity.

Figure 1 illustrates the page migration process managed by the UVM driver. When a GPU program accesses a UVM-managed page that is not currently resident in GPU memory, the GPU generates a ①page fault at a 4 KB page granularity and records this event in its fault buffer. Upon receiving an interrupt triggered by these faults, the UVM driver retrieves fault records from the buffer and initiates the ②fault handling process. Although faults occur at a 4 KB granularity, the UVM driver batches and processes faults belonging to the same 2 MB VABlock together to enhance efficiency [30].

The fault handling procedure for each VABlock typically involves three sequential stages: ③ *Populate*, ④ *Eviction*, and ⑤ *Copy*. During the *Populate* stage, the UVM driver checks for available unused chunks in GPU memory and allocates a suitable chunk to the page-faulted VABlock. Typically, since a chunk and a VABlock both have a fixed size of 2 MB in UVM-enabled systems, each VABlock is allocated exactly one chunk. If there are no unused chunks available, the *Eviction* stage is triggered, where the UVM driver selects and evicts the earliest accessed chunk from the GPU. The data of this evicted chunk is migrated back to host memory, freeing space for the incoming allocation. In the subsequent *Copy* stage, the driver copies data from the host memory into the allocated GPU chunk. Although GPU memory allocation during the *Populate* stage is performed at a 2 MB granularity, the actual data copying typically occurs at a smaller granularity than 2 MB to ensure efficiency. Instead of copying pages individually



(a) Latency breakdown of UVM page fault handling. (b) Slowdowns under different memory oversubscription ratios.

Fig. 2: Characteristics of UVM and Access-counter based migration UVM (AC) under memory oversubscription.

(4KB each), the driver commonly performs copies at larger intermediate granularities (e.g., 64 KB) and simultaneously selects adjacent pages for prefetching to further improve transfer efficiency [16], [34]. This host-side fault handling mechanism, which involves long-latency operations and coarse-grained memory management, can significantly degrade performance, particularly under high memory pressure.

III. DIAGNOSIS OF EXISTING WORK

A. Challenges in the Existing UVM Systems

Page Fault Handling Overheads. To improve efficiency, the UVM driver fetches up to 256 page faults at once from the GPU fault buffer [4]. Since these faults often originate from multiple VABlocks, handling the entire batch requires completing the fault handling process for all involved VABlocks [2]. Despite batching and VABlock-level grouping, handling the faults for a single VABlock typically takes tens of microseconds [30]. As a result, processing a full batch may take several milliseconds—latency that cannot be hidden by GPU context switching, leading to noticeable performance degradation [4], [5], [32], [58].

Figure 2a illustrates the average latency of the three key VABlock fault handling operations, *Populate*, *Eviction*, and *Copy*, measured across 10 different benchmarks in Table I. The results confirm that the *Populate* operation incurs the most significant latency, which is nearly twice that of the *Copy* and *Eviction* operations. This extended duration is primarily caused by the host-side `unmap` operation [2], an unavoidable step before pages can be copied to the GPU. To address the long fault handling time, prior works [30], [32] have attempted to overlap *Eviction* and sequential *Populate-Copy* by leveraging the two DMA copy engines available on modern GPUs [41]. However, since *Populate* alone dominates the total latency as we observed, such parallelization yields limited overall benefit. Consequently, hiding or reducing *Populate* latency is essential for meaningful UVM performance improvement.

Thrashing. When the working set size exceeds the available GPU memory, recently evicted pages are likely to be accessed again soon afterward, triggering repeated page faults and forcing their migration back into GPU memory. This repeated eviction and reloading of active chunks, commonly referred to as *thrashing*, causes the GPU to spend the majority of its

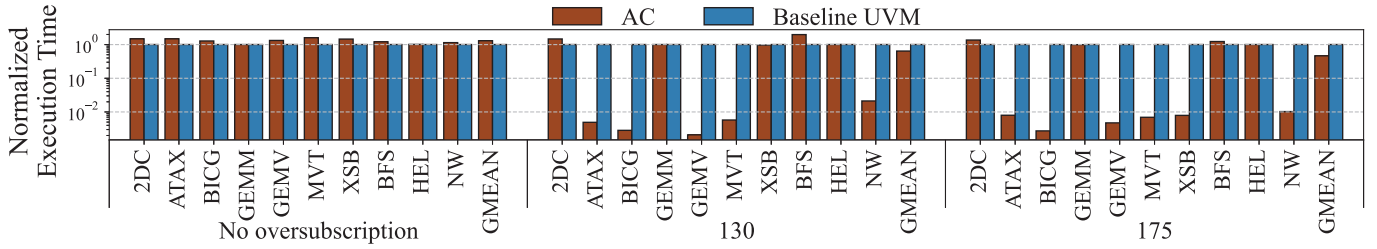


Fig. 3: The execution time of Access counter-based UVM (AC) and Baseline UVM.

time handling faults and transferring data rather than executing kernel computations. For clarity, in this paper, we define the aggregate size of GPU chunks actually demanded by a UVM system as the Working Chunk Set Size (WCSS). Note that WCSS can be significantly larger than the actual memory footprint of the workload, since GPU memory is allocated at a coarse granularity (e.g., 2 MB chunks), which may result in internal fragmentation.

Figure 2b illustrates the normalized geomean performance of the baseline UVM system and the Access counter-based migration (AC) across the applications in Table I, under 100%–300% oversubscription ratios. As shown in the figure, baseline UVM’s execution time increases by $33.2\times$ when the memory footprint is $2\times$ the available GPU memory, and by $60.7\times$ when it reaches $3\times$.

While such slowdowns are a consequence of thrashing, several characteristics of UVM, discussed next, make the system susceptible to thrashing. First, UVM’s coarse-grained 2 MB chunk allocation inflates memory demand even when only a few 4 KB pages are needed. This mismatch between actual usage and allocation becomes especially problematic for sparse access patterns, further increasing the risk of thrashing [32], [48]. Second, when GPU memory becomes scarce, UVM selects victims primarily based on their last fault time. This simplistic replacement policy often evicts active VABlocks, causing them to be re-fetched almost immediately. Such repeated eviction–refetch cycles not only waste valuable CPU–GPU bandwidth but also intensify thrashing by displacing useful data and triggering additional page faults. Lastly, UVM’s long fault handling latency and speculative prefetcher exacerbate thrashing [12]. The lengthy handling process penalizes the repeated memory copies inherent to thrashing, while speculative prefetching [16], [20] further increases data movement, extending fault resolution time. As a result, thrashing in UVM systems not only arises more readily under high memory pressure but also incurs substantial overhead due to the additional data transfers over the low-bandwidth CPU–GPU interconnect.

Summary: UVM suffers from long page fault handling latency, with *Populate* being the largest contributor. It is further degraded by severe thrashing under memory pressure, where coarse-grained allocation and last-fault-time victim selection frequently re-evict needed data, causing repeated refetches and excessive CPU–GPU transfers.

B. Prior Approaches for Thrashing Mitigations: Adjust Page Placement via Zero-copy

To prevent thrashing in UVM systems, prior works have utilized Zero-copy: a technique where pages are pinned in host memory, allowing the GPU to directly access them at cache-line granularity without migration. A prominent real-machine technique is NVIDIA’s Access counter-based migration (AC) [41]. AC initially places all pages in a Zero-copy state, then uses access counters to migrate any page to the GPU whose access count surpasses a static threshold. By copying only the most recently accessed pages to the GPU, this AC solution lowers the overall page fault rate, thereby mitigating thrashing. Even though Zero-copy accesses have substantially lower transfer bandwidth than on-board GPU memory accesses, the performance gain from mitigating thrashing can exceed these overheads.

Figure 3 presents the results for 10 benchmarks executed with both Baseline UVM and AC under three scenarios: no oversubscription, 130% oversubscription, and 175% oversubscription. In this figure, AC outperforms the baseline UVM by an average of $2.2\times$ and $150\times$ on thrashing-prone applications, at 175% oversubscription, highlighting the potential of Zero-copy for mitigating thrashing.

However, AC’s static threshold-based page placement, which disregards memory access patterns and the current GPU memory pressure, causes performance degradation in scenarios where memory pressure is either lower or higher than the optimal spot. Under no oversubscription in Figure 3, the execution time for AC increases by an average of $1.3\times$ and up to $2.5\times$ compared to the baseline UVM. In addition, AC fails to completely suppress thrashing under high memory oversubscription ratios. In Figure 2b, the performance of AC degrades quadratically; at 210% and 300% oversubscription, the system is $4.7\times$ and $8.9\times$ slower, respectively, than the case without oversubscription.

Unlike AC, achieving optimal performance under varying GPU memory pressures requires dynamically adjusting page placement based on runtime memory conditions and access patterns to minimize the Zero-copy penalty. A key distinction guiding page placement is that Zero-copy pages are accessed at cache-line granularity (128 B) and loaded into the GPU cache, whereas migration occurs in 2 MB chunks. Due to this contrast, while a VABlock with sparse access would cause significant WCSS amplification if migrated, with Zero-copy access, the penalty is minimized because only actively

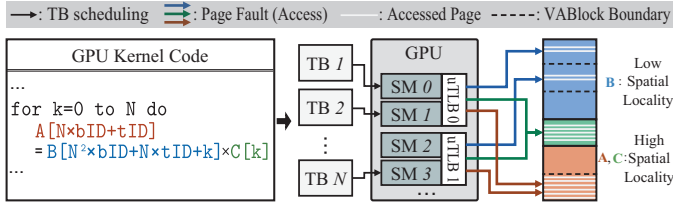


Fig. 4: GPU thread execution architecture.

used data is transferred at a fine, cache-line granularity and subsequently cached into a few cache lines. Conversely, a VABlock with high utilization where all internal pages are accessed would experience no WCSS amplification upon migration but would suffer a significant performance degradation with Zero-copy due to the need for remote access for every page. This intuition suggests that dynamically adjusting the Zero-copy policy based on a VABlock's spatial locality (i.e., VABlock utilization) can lead to a system-level optimized page placement.

Leveraging this intuition, several prior works [7], [9], [33] have proposed techniques to selectively orchestrate page placement between GPU memory and Zero-copy based on access patterns. However, these approaches require hardware modifications or compiler assistance, which undermines the core advantages of UVM: application portability and GPU memory abstraction. The reliance of prior works on additional modifications stems from a limitation of the UVM system: the host UVM driver can only observe GPU memory accesses through page faults. While some study [20] has used page fault addresses and timestamps to infer access patterns at runtime, this metric becomes inaccurate once thrashing is mitigated and faults become infrequent. In such minimum-fault scenarios, the technique cannot measure spatial locality, rendering it unsuitable for the dynamic Zero-copy adjustments required to maintain a stable, thrashing-free state. Therefore, a new runtime metric is required that can quantify spatial locality even in minimum-fault conditions.

Summary: A promising approach to mitigate thrashing is to dynamically adapt VABlock placement based on memory access sparsity, configuring sparse VABlocks for Zero-copy and migrating dense VABlocks, at runtime.

IV. OPPORTUNITIES FOR UVM PERFORMANCE ENHANCEMENT

A. Sharing Degree: An Effective Runtime Metric for GPU Memory Locality

We propose the **Sharing Degree** as a runtime metric representing the number of SMs (via their uTLBs) concurrently accessing a VABlock. Notably, a high Sharing Degree inherently implies that a given VABlock possesses high spatial locality, a characteristic rooted in the GPU software/hardware thread execution model. Furthermore, the Sharing Degree, which can be measured with only a few faults, is well-suited for guiding the dynamic Zero-copy page placement decisions for thrashing

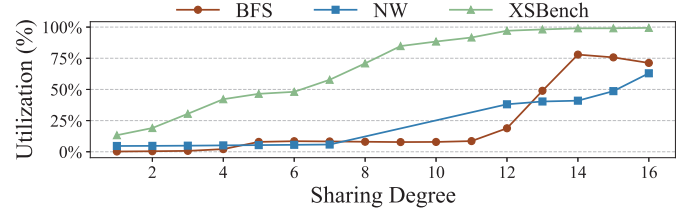


Fig. 5: The VABlocks' Sharing Degree influences the number of pages accessed before eviction.

mitigation. In this section, we analyze this correlation between Sharing Degree and memory access patterns, both theoretically and empirically.

As analyzed in §II-A, addresses accessed by threads in a GPU program change with the coefficients of the tID and bID (e.g., $Array[N \times bID + M \times tID]$) [7], [29], [51]. Figure 4 illustrates how memory regions are accessed through both the GPU's software and hardware thread execution architecture. For data **B**, the large coefficient of the bID term (e.g., N^2) causes each TB to access a distinct VABlock. As a result, the SMs executing these TBs also access different VABlocks, leading to sparse, low-locality accesses. Conversely, for data **A**, **C**, as the coefficient of the bID N is small, adjacent TBs will access the same page and VABlock, leading to concurrent access to these locations by the SMs. Furthermore, due to each TB's bID difference and the varying tID during execution, SMs concurrently accessing the same VABlock will access different addresses and distinct internal pages. As a result, from the perspective of a VABlock, the more SMs that access it concurrently, the more internal pages are accessed simultaneously, resulting in dense access (high VABlock-level spatial locality) and utilization. The Sharing Degree is designed to capture these fundamental memory access patterns from the GPU architecture by tracking thread access information on a per-VABlock basis.

We validate these insights by measuring the utilization of VABlocks as a function of their Sharing Degree in mixed-pattern workloads. Figure 5 shows the VABlock-level utilization according to the Sharing Degree for benchmarks with mixed access patterns: BFS, NW, and XSB [7], [20]. The Sharing Degree was measured using the method described in §V-B, and utilization is defined as the percentage of pages accessed within a VABlock between its migration and eviction. In all these three mixed workloads, the average number of pages accessed within a VABlock consistently increases with the Sharing Degree. This experimental result thus substantiates our insight, derived from the analysis of the GPGPU thread execution architecture, regarding the correlation between Sharing Degree and spatial locality.

Consequently, the Sharing Degree effectively differentiates between distinct memory access patterns at runtime. Figure 6 illustrates the runtime measurements of the Sharing Degree for each page fault across six benchmarks. Each point represents a single fault; the x -axis indicates the time of the fault, the y -axis represents its virtual address, and the color of the point

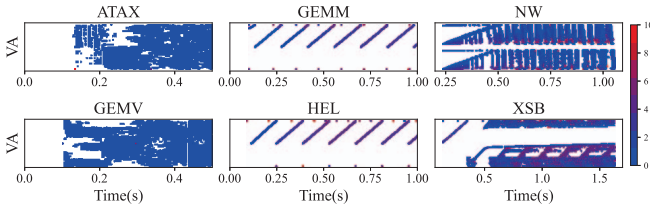


Fig. 6: The memory access patterns of six benchmarks.

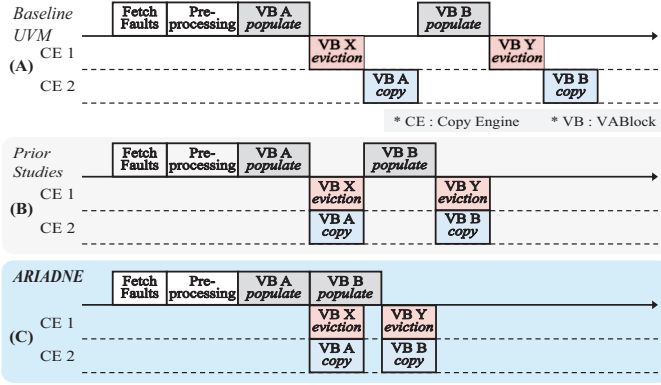


Fig. 7: Pipelined fault handling operations.

indicates the Sharing Degree of the VABlock where the fault occurred.

ATAX and GEMV, representative benchmarks previously categorized as irregular and sparse access patterns [32], consistently show a Sharing Degree of 1. In contrast, GEMM and HEL, which have been analyzed as regular and dense access patterns [20], exhibit a Sharing Degree that is generally greater than 4. NW and XSB are known to have mixed patterns [15] with various access types coexisting. The measurement of their Sharing Degree also reveals that VABlocks with contrasting values are accessed simultaneously. This further emphasizes the need to distinguish access patterns at the VABlock level, rather than tracking characteristics at the level of the entire GPU or individual CUDA kernels, in order to accurately measure locality on a per-VABlock basis.

Insight 1: The Sharing Degree captures the inherent spatial locality of a memory region by quantifying how many SMs access it concurrently. This runtime metric effectively distinguishes between dense and sparse access patterns, enabling informed GPU memory management decisions.

B. Hiding Populate Latency via Pipelined Fault Handling

Although the *Populate* operation incurs nearly double the latency of *Copy* and *Eviction*, the prior works [30], [32] fail to hide this overhead as they do not decouple *Populate* from the original copy process. Therefore, we explore decoupling the *Populate* from the original copy process and enabling concurrent execution with *Copy* and *Eviction*. As a result, we found that while *Copy* and *Eviction* for a single VABlock's fault handling must be performed after *Populate*, *Populate*,

Copy, and *Eviction* operations for different VABlocks can indeed be executed simultaneously.

Figure 7 illustrates the execution flow of the VABlock fault handling process under different implementations. (A) represents the baseline UVM, where the *Populate*, *Eviction*, and *Copy* stages of the VABlock fault handling process are performed sequentially. (B) illustrates the approach proposed in prior studies [30], [32], in which *Copy* and *Eviction* are performed concurrently. (C) depicts the pipelined VABlock fault handling process proposed in this study. In (C), after VABlock A completes its *Populate*, the *Populate* of VABlock B, the *Eviction* of VABlock X (which holds the eviction victim chunk), and the *Copy* of VABlock A are performed concurrently. As a result, we can hide a significant portion of populate's latency by pipelining VABlock fault handling.

Insight 2: Decoupling *Populate* from subsequent *Copy/Eviction* allows pipelined execution across multiple VABlocks, enabling substantial overlap between stages and effectively hiding the long *Populate* latency.

V. DESIGN OF ARIADNE

We propose an adaptive UVM management for efficient GPU memory oversubscription, called ARIADNE. By recognizing the real-time memory demands and optimally adjusting the placement of VABlocks based on Sharing Degree, ARIADNE delivers efficient performance across diverse workloads and memory oversubscription ratios. We implement ARIADNE by modifying the UVM host driver (~ 1600 LOC), requiring no additional GPU resources or architectural modifications over the baseline UVM. Figure 8 illustrates the high-level design of ARIADNE, highlighting its core components.

A. Working Chunk Set Size (WCSS) Estimation

To maximize GPU memory utilization while preventing thrashing, accurately tracking the current memory demand of the UVM system and precisely managing the excess demand via Zero-copy placement in host memory is crucial. Since applications request GPU memory in chunk units (e.g., 2 MB), ARIADNE tracks the WCSS, representing the number of GPU memory chunks actively required by workloads. Given that each VABlock corresponds to exactly one chunk, ARIADNE calculates WCSS by counting all actively accessed VABlocks. Ideally, this includes VABlocks currently resident on the GPU and those in a Zero-copy state on the host memory, as ensured by ARIADNE's dynamic Zero-copy policy (§V-D).

However, this straightforward calculation can underestimate the actual WCSS if a VABlock that is likely to be used again soon is evicted from GPU memory, as it is immediately removed from the WCSS count despite its high probability of being re-accessed in the near future. To resolve this, ARIADNE maintains a per-VABlock re-access history upon eviction. A recently evicted VABlock that is re-accessed remains included in the WCSS for 500ms after eviction. Additionally, when no re-access history is initially available, ARIADNE measures the average GPU-wide Sharing Degree. This is because, as

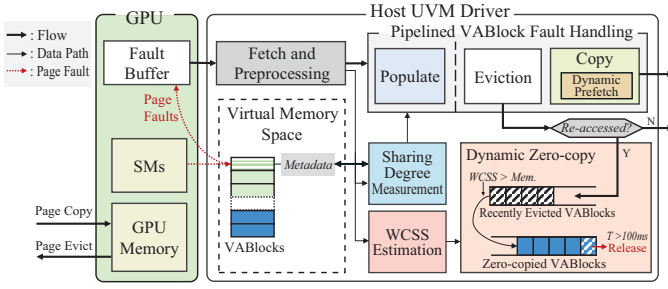


Fig. 8: Overall design of ARIADNE.

discussed in §IV-A, applications with a low average Sharing Degree encounter chunk-level internal fragmentation, exhibit amplified WCSS, and are susceptible to initial thrashing. Thus, if this average Sharing Degree is below a predefined threshold (i.e., 3), all recently evicted VABlocks are retained in the WCSS for 500 ms, thereby mitigating the risk of premature exclusion and initial thrashing.

Consequently, ARIADNE accurately estimates the WCSS at any given moment as the sum of: 1) VABlocks resident in GPU memory, 2) VABlocks currently in a Zero-copy state, and 3) recently evicted VABlocks with a high probability of re-access.

B. Sharing Degree Measurement

ARIADNE tracks the Sharing Degree, a metric strongly correlated with access sparsity, at a per-VABlock granularity within the UVM kernel module driver (`nvidia-vm`) [40]. To achieve this, ARIADNE leverages the page fault's source uTLB ID, which serves as a reliable proxy for identifying the originating SM and, consequently, the TB responsible for the fault. Specifically, the host UVM driver maintains a circular queue that records the source uTLB IDs of the most recent 16 faults for each VABlock. The Sharing Degree is then computed as the number of unique uTLB IDs currently present in this queue, and this value is updated at the beginning of the fault handling process for each VABlock. Importantly, as the source uTLB ID is readily available to the UVM driver during runtime, the overhead of calculating the Sharing Degree is negligible. This metric effectively captures the fundamental memory access patterns arising from the GPU's thread execution structure, enabling ARIADNE to obtain critical runtime information efficiently without intrusive or costly profiling techniques, as discussed in §IV-A.

C. Enhanced VABlock Fault Handling

In baseline UVM, VABlock fault handling (Figure 7a) is performed sequentially within a monolithic routine that includes the *Populate*, *Eviction*, and *Copy* phases. This serialized design forces high-latency operations such as GPU memory allocation and page copying to execute back-to-back, prolonging fault handling latency. Based on insights §IV-B, a promising solution is to pipeline the fault handling process across consecutive VABlocks, enabling the *Populate* stage to execute in parallel with other operations.

Figure 7c illustrates how ARIADNE restructures this process into a two-stage pipeline. Stage1 (*Populate*) allocates GPU chunks and prepares migration metadata, while Stage2 (*Copy*) transfers the corresponding pages to the GPU. Unlike the baseline, these two stages can operate concurrently on different VABlocks, enabling higher throughput. In addition, *Eviction* is no longer a reactive step following a failed *populate*, but rather a proactive process running in parallel to ensure a steady supply of free chunks. By decoupling these phases and overlapping their execution, ARIADNE effectively hides much of the high-latency operations behind parallel work.

ARIADNE further enhances this pipelined framework by integrating a dynamic prefetcher into *Copy* and a Sharing Degree-aware priority eviction queue into *Eviction*. The prefetcher reduces future page faults by proactively loading VABlocks with high reuse potential, while the priority eviction policy retains such VABlocks in GPU memory for longer.

Populate. The *Populate* unmaps the target host pages, allocates a GPU chunk for the VABlock, and records the pages to be migrated. Unlike the baseline, ARIADNE terminates *Populate* without initiating the page copying, allowing the allocation to complete quickly and enabling the subsequent *Copy* to be pipelined with the next *Populate* request.

Copy. The *Copy* runs in parallel with *Populate*, transferring the pages of a populated VABlock to GPU memory using the GPU copy engine and updating page tables. By default, ARIADNE copies the pages requested along with those selected by the default prefetcher (e.g., TBN prefetcher [41]). If GPU memory usage is sufficient or the VABlock's Sharing Degree exceeds a threshold (three in our design), ARIADNE aggressively copies the entire VABlock. This dynamic prefetching reduces future page faults by proactively bringing in pages likely to be reused before eviction.

Eviction. Rather than waiting for *Populate* to fail, ARIADNE initiates *Eviction* when free GPU chunks drop below a threshold (one in our design). *Eviction* runs concurrently with *populate* and *copy* in a dedicated thread. To avoid evicting actively used VABlocks, ARIADNE employs a priority queue that incorporates both fault recency and Sharing Degree:

$$key = last_fault_time + \left(SD_Weight \times \frac{Sharing\ Degree}{N_{fault_history}} \right)$$

where $N_{fault_history}$ denotes the length of the history tracking uTLB IDs that recently caused faults (16 in our implementation), and SD_Weight (set to $100\mu s$ based on sensitivity analysis in §VII-E) adjusts the influence of Sharing Degree. This approach enables ARIADNE to retain data with a high Sharing Degree—for which GPU residency is more advantageous than Zero-copy, as discussed in §IV-A—on the GPU for longer durations, thereby reducing costly re-fetches.

D. Dynamic Zero-copy

When the workload demands more memory than the available GPU memory, ARIADNE dynamically places certain VABlocks into a temporary Zero-copy state to both reduce

internal fragmentation in GPU memory and mitigate the performance penalty of frequent migrations. The key idea is to keep recently evicted VABlocks that are re-accessed, those counted in the WCSS measurement (§V-A), in a Zero-copy state rather than immediately fetching them back to GPU memory. This second-chance mechanism prevents immediate re-fetch of VABlocks that were just evicted, thereby reducing repeated eviction–fetch cycles and alleviating thrashing.

Evicted VABlocks are determined by the Sharing Degree–aware priority eviction queue (§V-C), which prioritizes eviction of VABlocks with low Sharing Degree and low predicted reuse potential. However, if any of these evicted VABlocks are subsequently accessed again, ARIADNE places them in the Zero-copy state for a fixed duration (100ms in our design) instead of bringing them back to GPU memory immediately. Since these VABlocks are already resident in host memory, enabling Zero-copy requires no data copying and incurs negligible overhead. If a Zero-copied VABlock is accessed again after the expiration period, it is then promoted back into GPU memory, ensuring that persistently reused VABlocks eventually regain GPU residency while minimizing unnecessary memory churn.

The Zero-copied VABlocks are tracked in a dedicated Zero-copy queue and automatically released from Zero-copy state after the predefined duration. This release step is essential for accurate WCSS measurement and for maximizing GPU memory utilization, since the UVM driver relies on page faults to detect activeness. If Zero-copy state were permanent, inactive VABlocks could indefinitely hold GPU chunks, blocking more beneficial allocations. Conversely, setting the duration too short would limit thrashing prevention. Empirical analysis confirms that our chosen 100ms duration strikes a good balance across diverse workloads and memory pressures (§VII-E).

This mechanism is implemented using internal APIs of the UVM kernel module driver. We implement functions to set and unset the Zero-copy state for target VABlocks. Metadata for this management is maintained on a per-GPU basis within the UVM driver without consuming GPU memory.

E. Putting it all together

When a UVM kernel launches, ARIADNE initializes all required metadata and runtime objects, including dedicated kernel threads for *Copy* and *Eviction*. When a GPU memory access triggers a page fault, the faults are collected in the GPU fault buffer and fetched by the UVM driver. The driver preprocesses these faults and processes them on a per-VABlock basis through ARIADNE’s pipelined fault handling mechanism. During this process, ARIADNE updates both the WCSS and the Sharing Degree using the observed fault information.

After handling all fetched faults, ARIADNE checks whether the memory demand by workloads exceeds the available GPU memory capacity. If so, ARIADNE places all recently evicted VABlocks that have been re-accessed into the Zero-copy state,

providing them with a second chance to be reused without immediately occupying GPU memory. If any VABlock remains in the Zero-copy state beyond the predefined duration (100ms in our design), ARIADNE revokes its Zero-copy status.

A VABlock in ARIADNE can be in one of four states: (1) resident on GPU, (2) stored in host memory but in Zero-copy mode, (3) evicted from GPU but subsequently re-accessed, and (4) evicted and not re-accessed. Based on its measured Sharing Degree and recent access history, a VABlock transitions between these states so that it remains in the one most advantageous for its current usage pattern. This continuous evaluation ensures that VABlocks likely to be reused are retained or given a Zero-copy second chance, while inactive ones are evicted promptly. By combining pipelined fault handling, Sharing Degree–aware eviction, and dynamic Zero-copy, ARIADNE adaptively mitigates thrashing and delivers robust performance across diverse workloads and oversubscription ratios.

VI. DISCUSSIONS

Fine-grained Chunk Size Since UVM uses a coarse-grained 2 MB chunk size, one possible approach to thrashing mitigation is to adopt fine-grained chunks to mitigate WCSS amplification. In our experiment, while fine-grained chunk size can mitigate thrashing in certain workloads with frequent sparse access (ATAX, BICG, GEMV, MVT), it causes significant performance degradation (64.4%) in others (2DC, GEMM, XSB, BFS, HEL). The performance degradation is caused by two major factors: a reduced TLB reach and the overhead from repeatedly searching for GPU free space and *Populate* operation for every 64 KB chunk allocation. Moreover, the smaller chunk approach is not a scalable solution because thrashing is fundamentally unavoidable when the actual byte-level working-set size exceeds the GPU physical memory capacity. Furthermore, while a dynamic chunk sizing technique combined with the Sharing Degree has the potential to minimize the overhead of fine-grained chunks, a naïve implementation causes severe external fragmentation, and necessitates precise and intelligent chunk management. Therefore, ARIADNE does not adopt a fine-grained chunk size.

VII. EVALUATION

A. Experiment Setup

System Configurations We evaluate the performance of ARIADNE on a real system which comprises an NVIDIA RTX A5000 GPU and an AMD Ryzen 7700X CPU, connected via a PCIe 4.0 x16 interface, with 64 GB of DDR5 DRAM. We use Linux kernel version 6.0 and the NVIDIA open-source kernel driver version 535.86. For experiments involving memory oversubscription, we reserve a portion of the GPU memory using `cudaMalloc`, a common method in prior studies [7], [20], [22]. In evaluation, the oversubscription ratio is calculated as: $\frac{\text{Memory Footprint}}{\text{GPU Memory Size}}$. To validate ARIADNE under various memory conditions—from ample to moderate and severe pressure—we conduct experiments under no oversubscription, as well as 130%, 175%, and 300% oversubscription ratios.

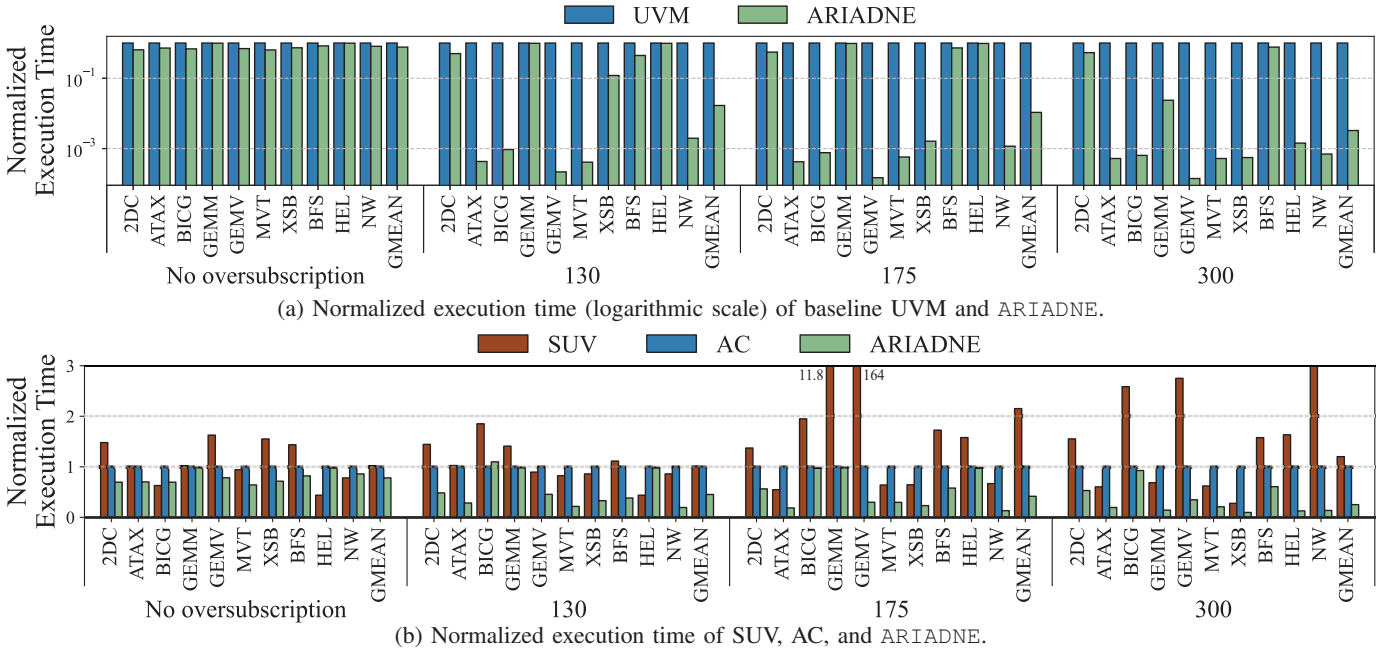


Fig. 9: Overall performances.

Abbr.	Benchmark
2DC	2D convolution of matrix
ATAX	Matrix transpose and vector multiplication
BICG	BiCG sub kernel of BiCGStab linear solver
GEMM	Matrix multiplication
GEMV	Scalar, vector and matrix multiplication
MVT	Matrix vector product and transpose
XSB	XSBBench: Monte Carlo neutron transport algorithm
BFS	Breadth first search
HEL	Calculate hellinger distances
NW	Needleman-Wunsch algorithm

TABLE I: Benchmark Configurations.

Workload As shown in Table I, our experiments are conducted on 10 GPGPU benchmarks from the Rodinia [10], [22], [23], Polybench [21], [22], HeCBench [27], and XSBBench [49] suites, which feature diverse memory access patterns. The memory footprint for all benchmarks is set to 4 GB. Execution time is measured as the end-to-end runtime of the CUDA kernels included in the benchmarks.

B. Overall Performance of ARIADNE

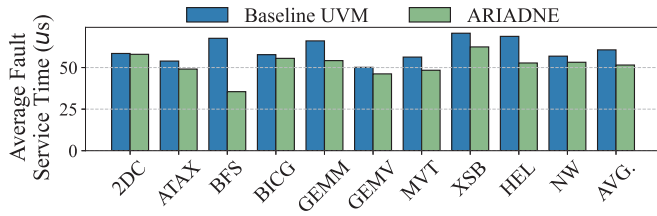
In this section, we compare the performance of ARIADNE against 1) the baseline UVM system and 2) two state-of-the-art (SOTA) real-system methodologies. The baseline UVM system utilizes the default NVIDIA open-source kernel module (v535.86). The first SOTA is NVIDIA's Access Counter-based migration (AC) [40]. The second is SUV [7], for which we apply its provided API to the benchmark codes, configure compile-time variables such as GPU free memory size and footprints, and recompile them using the SUV framework. Since the baseline UVM system suffers from thrashing, leading to a massive performance

degradation compared to SOTA methods, we first compare UVM and ARIADNE, and then evaluate ARIADNE against the SOTA methods, for clarity.

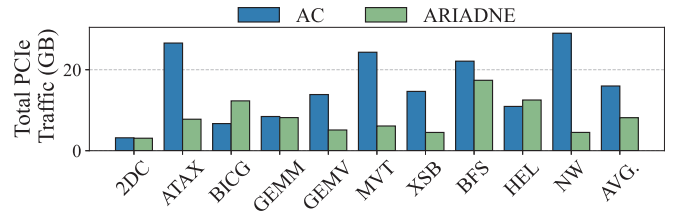
Compare with baseline UVM Figure 9a presents the execution time of UVM and ARIADNE across various benchmarks and oversubscription ratios, plotted on a logarithmic scale. Runtimes are normalized to that of the baseline UVM in each scenario. Across all tested benchmarks and oversubscription ratios, ARIADNE consistently demonstrates significantly shorter execution times than UVM. The primary reason for these improvements is ARIADNE's ability to prevent thrashing by dynamically placing VABlocks with low Sharing Degree into Zero-copy. Even under a 300% oversubscription, ARIADNE successfully suppresses thrashing for benchmarks with sparse access patterns like ATAX, GEMV, and MVT. Furthermore, for benchmarks like GEMM and HEL, which do not thrash at 175% but do at 300% oversubscription, ARIADNE effectively prevents thrashing. Moreover, ARIADNE outperforms the baseline UVM even in no-oversubscription scenarios, thereby validating the effectiveness of pipelined VABlock fault handling.

Compare with SOTAs ARIADNE also achieves substantial performance improvements over the SOTA methods, SUV and AC. Figure 9b shows the execution times of ARIADNE, SUV [7], and AC [41] normalized to AC across various benchmarks and oversubscription ratios.

At 130%, 175%, and 300% oversubscription, ARIADNE delivers geomean speedups of 1.9 $\times$, 2.3 $\times$, and 4.0 $\times$ over AC, and achieves geomean speedups of 1.9 $\times$, 5.0 $\times$, and 4.8 $\times$ over SUV. Crucially, ARIADNE's performance advantage over SUV and AC grows as the oversubscription ratio increases. This signifies that ARIADNE's design, which lever-



(a) Average VABlock fault handling latencies.



(b) Total PCIe traffic at 175% oversubscription.

Fig. 10: Speedup analysis.

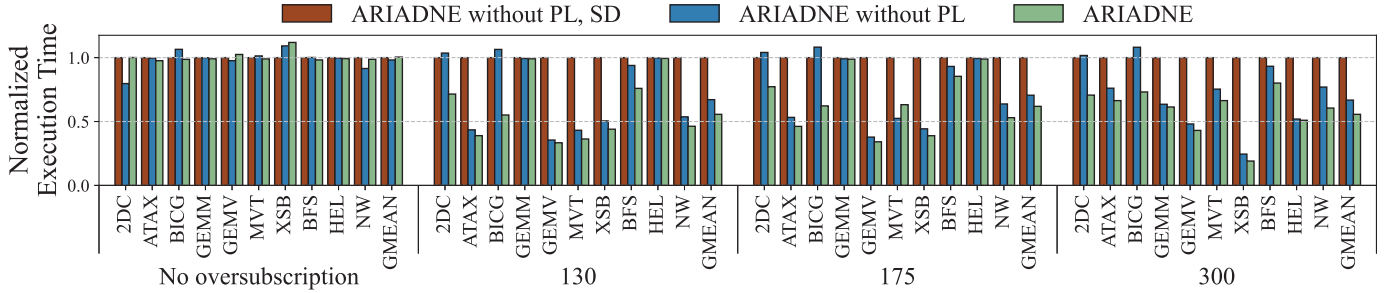


Fig. 11: Performance breakdowns.

ages Sharing Degree for runtime dynamic Zero-copy, prevents thrashing and achieves efficient page placement scalable to memory demands. Consequently, when the allocated memory is $1.3\times$ (130%), $1.75\times$ (175%), and $3\times$ (300%) the available GPU memory, ARIADNE's runtime is only $1.6\times$, $1.8\times$, and $2.3\times$ that of the no-oversubscription case, demonstrating linear performance degradation characteristics. This contrasts sharply with the exponential degradation of UVM and the quadratic degradation of AC and SUV, highlighting the scalability and practicality of ARIADNE's design for oversubscription.

ARIADNE consistently delivers performance gains over AC and SUV in mixed access patterns benchmarks, such as BFS, XSB, and NW. Similarly, in applications that are inherently prone to thrashing due to sparse access patterns like ATAX, GEMV, and MVT, ARIADNE shows a significant performance advantage. This result suggests that the dynamic design of ARIADNE is more effective at preventing thrashing than either SUV's or AC's static approaches. For GEMM and HEL, ARIADNE performs similarly to AC up to 175% oversubscription because the working set sizes of these two benchmarks are considerably smaller than their footprints. However, at 300% oversubscription where they begin to thrash, ARIADNE provides a significant performance boost through its optimal dynamic VABlock placement.

In a few cases with no oversubscription, ARIADNE exhibits lower performance than SUV. This is attributed to the ARIADNE prefetcher, which is limited to a 2MB VABlock granularity. In contrast, SUV performs prefetching across large data ranges via compile-time static code analysis, thus achieving the highest performance in BICG, HEL and NW.

In thrashing-prone benchmarks, SUV outperforms AC but is outperformed by ARIADNE. However, in some scenarios, SUV underperforms compared to AC. For instance, due to

inaccuracies in its compile-level analysis, SUV fails to prevent thrashing in the GEMV benchmark at 175% oversubscription, highlighting the limitations of relying solely on compiler-assisted static analysis. The working-set size and access density, estimated through static code analysis of SUV, fail to accurately represent the dynamic memory environment, which is constantly changing due to thread scheduling and phase shifts during application execution. This discrepancy between static estimates and runtime reality may grow with GPU memory pressure, and the resulting performance penalty from incorrect memory management decisions also increases. Moreover, because ARIADNE can accurately measure a VABlock's real-time access characteristics via its Sharing Degree, it provides superior performance over SUV and AC in most low-oversubscription benchmarks.

We evaluated ARIADNE on the inference of the Llama3.1 70B model, as depicted in Figure 12. ARIADNE delivers $4.2\times$ and $1.6\times$ speedups over the AC in the Decode and Prefill phases, respectively. The performance gain is more pronounced in the Decode phase, which is dominated by GEMV operations. This is consistent with the results in Figure 9b, where ARIADNE yields greater improvements for GEMV kernels compared to GEMM kernels.

C. Reasons for Performance Improvement

Figure 10a shows the average VABlock fault handling time of Baseline UVM and ARIADNE, across 10 benchmarks. ARIADNE's Pipelined VABlock fault handling consistently reduces the VABlock fault handling latency by an average of 17%, up to 48% (BFS), without introducing penalties. Figure 10b presents a per-benchmark comparison of the total PCIe traffic for ARIADNE and AC. Across 10 benchmarks, ARIADNE consumes, on average, only 51% of the PCIe traffic

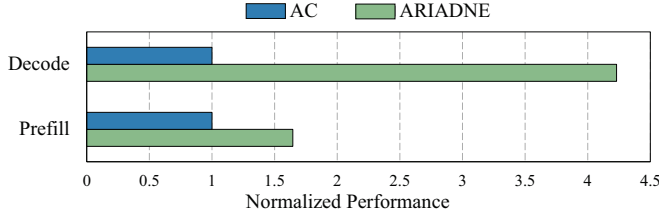


Fig. 12: Performances of Llama3.1 70B inference (input token length = 2048).

compared to AC. These results demonstrate the effectiveness of ARIADNE’s pipelined fault handling and dynamic VABlock placement policy, which is guided by the Sharing Degree.

D. Performance Breakdown

We investigate the performance contribution of each component of ARIADNE. Figure 11 compares the performance of ARIADNE without PL, SD (without Sharing Degree-based eviction queue and pipelined fault handling), ARIADNE without PL (without pipelined fault handling), and the full ARIADNE. At 175% oversubscription, ARIADNE without PL improves performance by 42.9% over ARIADNE without PL, SD, and the full ARIADNE achieves a 69% improvement over ARIADNE without PL, SD. These results demonstrate that a page placement policy guided by the Sharing Degree is the key player of ARIADNE’s performance gains, and that pipelining effectively hides the policy’s overhead under oversubscription.

E. Sensitivity Analysis

Zero-copy Pintime Figure 13a shows the performance variation with respect to the duration a VABlock remains in the Zero-copy state (i.e., Zero-copy Pintime). The results indicate that as long as the Pintime is not short enough to induce thrashing, the average performance difference is a negligible 1%. However, since extreme values led to significant performance degradation in some cases, we set the Pintime to a more stable 100 ms.

SD Weight Figure 13b presents the analysis for the SD Weight which is leveraged for Sharing Degree-aware eviction queue (§V-C). At 175% oversubscription, the execution time varies by 2.2% to 9% depending on the SD Weight. We set the SD Weight to 100 μ s, which provides the most stable and superior performance overall.

Moreover, Figure 13c demonstrates the geomean performance variation as both Pintime and SD weight are altered concurrently. These results confirm that the chosen combination of these parameters is optimal, revealing the performance to be comparatively more sensitive to changes in the SD weight.

GPU architecture Figure 13d illustrates the geomean performance slowdowns of ARIADNE and AC with respect to the memory oversubscription ratio across three different GPUs: the RTX 2060, RTX 3070, and RTX A5000. Across all tested GPUs, ARIADNE exhibits linear performance degradation,

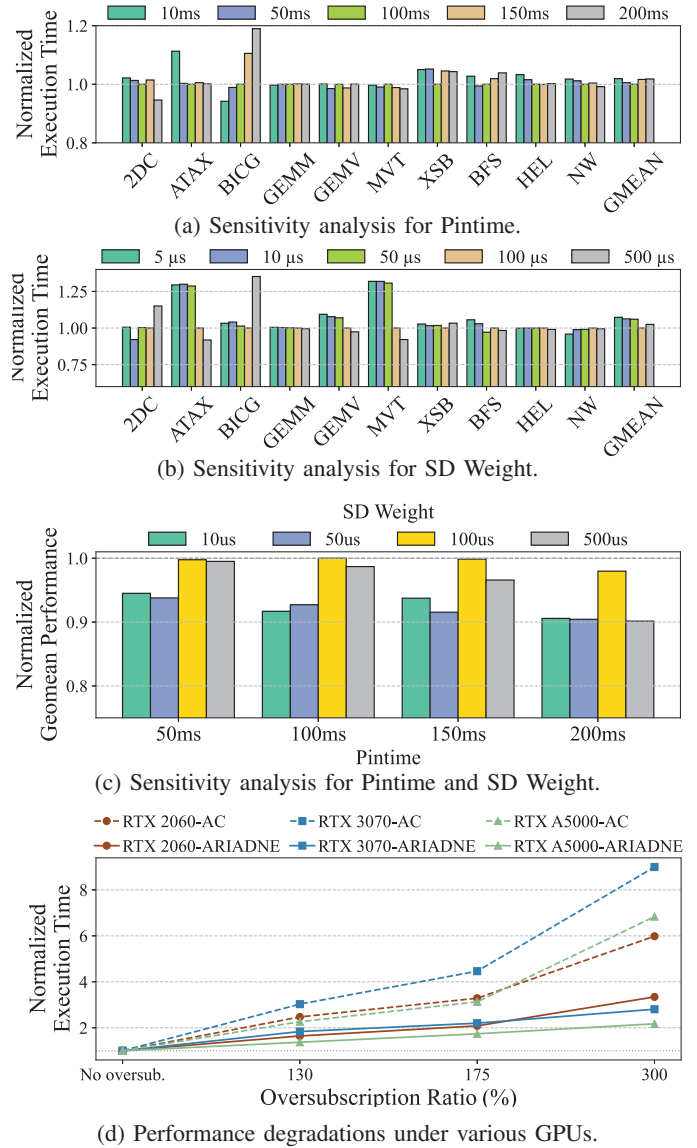


Fig. 13: Sensitivity analysis.

whereas AC shows quadratic degradation. Furthermore, this result validates that ARIADNE is applicable across various GPU architectures without modification.

F. Overheads

ARIADNE uses additional metadata to track Sharing Degree and perform dynamic Zero-copy at runtime UVM driver. Importantly, ARIADNE’s implementation requires no additional GPU resources. Also, as additional host driver metadata is less than 70 B per VABlock and less than 100 B per GPU, our implementation adds approximately 560 KB of memory for a 16 GB application; this spatial overhead is negligible given the tens of gigabytes of host memory.

The tracking and computation of Sharing Degree and WCSS, along with the Zero-copy process, introduce additional latency. However, our experiments show that these processes introduce a latency of at most 100 ns, which is negligible

within the context of the roughly 20 μ s fault handling process for a single VABlock. Consequently, ARIADNE's operations do not introduce any considerable overhead.

VIII. RELATED WORK

Characteristics of GPU UVM system Diverse studies have been conducted to analyze the performance characteristics of UVM systems, across various hardware and workloads [1]–[4], [11], [14], [16], [19], [29], [30], [45], [46], [48], [58]. Allen et al. [2], Gayatri et al. [19], and Zheng et al. [58] have validated the UVM's performance characteristics and effectiveness in various scenarios. Wang et al. [51], Vijaykumar et al. [50], and Jablin et al. [26] conduct in-depth analysis of the access patterns of GPU. Chien et al. [11], Agarwal et al. [1] and Shao et al. [46] evaluated the efficacy of various UVM memory policies. Kim et al. [30] analyze the inefficiency of fault handling process, and suggest thread oversubscription.

Memory management for GPU systems Various studies have been proposed to analyze the memory characteristics of GPGPU workloads and optimize its memory management policies. Prefetching prevents future faults and increases the copy efficiency by enlarging the data transferred at once [16], [19], [20], [28], [34], [36], [47], [54]. However, prefetchers alone cannot resolve thrashing when the actual working-set size exceeds GPU memory. Other studies have leveraged hardware [32], [44], [52] or code-level analysis [7], [9], [26], [29], [33], [35] to understand GPU's thread execution architecture-based memory access characteristics [29], [44] and adjust memory policies [5], [13], [24], [25], [31], [37], [38], [44], [44], [53], [55], [59] and residencies [7]–[9], [42]. For instance, LAMAR [44] identified the inefficiencies caused by fragmentation that depend on the locality of GPGPU workloads and proposed an effective method to dynamically determine the optimal data granularity. Dynamap [9] inserts additional instructions to utilize memory access information, while Li et al. and SUV [7] perform detailed static analysis at the code level. ETC [32], Choukse et al. [13], and Nihaal et al. [38] increase the effective memory size by compressing cold data on the GPU. Kim et al. [31] and ETC [32] introduce thread throttling to prevent thrashing by limiting memory demands below memory capacity. Notably, ARIADNE's design is distinguished from previous work by its ability to be implemented solely through kernel module modifications, without requiring any changes to hardware, the compiler, or application code. By leveraging the GPU thread execution architecture, ARIADNE quantifies the migration suitability as a Sharing Degree at runtime, then dynamically adjusts the page to its optimal location.

IX. CONCLUSION

By analyzing both the GPU's hardware and software thread execution structures, we devise the Sharing Degree, a metric of spatial locality of pages within the UVM driver. Leveraging the Sharing Degree, we propose ARIADNE, a dynamic runtime UVM management system. Through optimal page placement, ARIADNE achieves an average performance improvement of

$2.91\times$ over two SOTA methods and exhibits linear performance degradation even under significant GPU memory oversubscription. Notably, to the best of our knowledge, ARIADNE is the first runtime-only UVM management framework that requires no modifications to the hardware, compiler, or application code. Unlike prior approaches, ARIADNE preserves the GPU abstraction of UVM, making ARIADNE directly applicable to binary, closed-source applications.

ACKNOWLEDGEMENTS

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (RS-2025-25433771), Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2018-II180503, RS-2024-00396013, RS-2024-00459797, RS-2025-09942968, and RS-2025-02263869).

APPENDIX

A. Abstract

This artifact comprises ARIADNE, SOTA solutions for comparison, and the associated GPGPU benchmark suites, such as polybench, rodinia, hecbench, and XSBench. ARIADNE is implemented exclusively through modifications to the kernel module driver. The comparative SOTA, AC, refers to the access-counter based migration mechanism within the NVIDIA UVM driver. Additionally, SUV, a SOTA of compiler-assisted method, is included as an optional component, as it requires a different environmental configuration compared to ARIADNE. All basic experiments are executed via shell scripts, allowing for the reproduction of per-benchmark execution times for each configuration, as well as Figures 9, 11, and 13. Given that the results are normalized comparisons, variations in absolute execution times on different systems may alter the exact magnitude of performance gaps. Nevertheless, the general trends observed in the paper remain valid. The artifact is publicly available on Zenodo(<https://doi.org/10.5281/zenodo.17829999>).

B. Artifact check-list (meta-information)

- **Compilation:** GCC, NVCC
- **Data set:** Polybench, Rodinia, Hecbench, XSBench
- **Run-time environment:** linux 6.0, CUDA 12.1
- **Hardware:** NVIDIA RTX A5000
- **Run-time state:** NVIDIA open-kernel modules driver
- **Execution:** Shell scripts
- **Metrics:** Benchmark execution time
- **Output:** Figures, raw data
- **Experiments:** Final result reproduction
- **How much disk space required (approximately)?:** 15 GB (210 GB for comparison with SUV [7])
- **How much time is needed to prepare workflow (approximately)?:** 30 minutes (6 hours for comparison with SUV [7])
- **How much time is needed to complete experiments (approximately)?:** 2 hours
- **Publicly available?:** Yes
- **Workflow automation framework used?:** Shell scripts
- **Archived (provide DOI)?:** Yes. (10.5281/zenodo.17829999)

C. Description

1) *How to access*: All related files are archived and available on Zenodo. (<https://doi.org/10.5281/zenodo.17829999>)

2) *Hardware dependencies*: NVIDIA A5000 GPU, 15 GB of free disk space. (120 GB of main memory and 210 GB of free disk space is required to produce the result of SUV [7]).

3) *Software dependencies*: Linux kernel 6.0, CUDA 12.1, NVIDIA open-flavor kernel module driver v535.86. (Linux 6.2, CUDA 11.8, NVIDIA open-flavor kernel modules driver v525 are required for SUV [7] experiments).

4) *Data sets*: Polybench, Rodinia, Hecbench, XSBench.

D. Installation

Root privileges are recommended.

1. make clean on the /ARIADNE, /UVM directory.
2. In benchmarks/, 'nvcc hostpin.cu -o hostpin'
3. In benchmarks/, 'bash compile.sh'

The 'base directory' refers to the root folder of this artifact, specifically HPCA_AE_ARIADNE. The kernel modules for ARIADNE and AC will be automatically built and installed during the execution of the experiment scripts provided below. If you prefer to build them manually, please use Install_ARIADNE.sh and Install_AC.sh

E. Experiment workflow

Root privileges are recommended. Perform the below instructions on an environment where video kernel modules (nvidia.ko, nvidia-uvmm.ko, nvidia-drm.ko, and nvidia-modeset.ko) are not in use.

1. In base directory, 'bash run_ARIADNE_AC.sh'
2. In base directory, 'bash run_breakdown.sh'
3. In base directory, 'bash run_sstv_analysis.sh'
4. In results/, run all three Python scripts.

F. Evaluation and expected results

Key experimental results corresponding to Figures 9, 11, and 13 can be found in the result/ directory as raw data and graph images.

Figure 9: Performance Comparison Comparison of the geometric mean execution time of ARIADNE against AC (and SUV) across various workloads and oversubscription levels.

Figure 11: Component breakdown Analysis Illustrates the relative contributions of ARIADNE's key components.

Figure 13: Parameter Sensitivity Shows normalized geometric mean performance across different parameter settings.

Please note that while exact numerical values and ratios may vary due to differences in absolute execution times across real-system configurations, the trends remain consistent.

G. Experiment customization

If the GPU memory capacity differs from the experimental environment, the memory reservation amount in run_bench.sh must be adjusted accordingly.

H. Note

The comparison with SUV is provided as an optional component in this artifact, as it requires a different system configuration (specifically, different GPU driver and CUDA versions). If you wish to run the SUV experiments, please first configure your system environment accordingly and then execute run_SUV.sh located in the base directory. We utilized the provided artifact of SUV [7]; for detailed information, please refer to its paper and artifact documentation [6].

Please note that ARIADNE is a research prototype developed for experimental validation, not a commercial product intended for general deployment. Consequently, rare fatal bugs may occur. In such cases, please attempt to re-execute the experiment or reboot the system before retrying. Furthermore, while the design and concepts of ARIADNE are capable of supporting multi-GPU environments, the research prototype provided in this artifact has not been implemented or tested for such configurations.

I. Methodology

Submission, reviewing and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- <https://cTuning.org/ae>

REFERENCES

- [1] N. Agarwal, D. Nellans, M. Stephenson, M. O'Connor, and S. W. Keckler, "Page placement strategies for gpus within heterogeneous memory systems," *SIGPLAN Not.*, vol. 50, no. 4, p. 607–618, Mar. 2015. [Online]. Available: <https://doi.org/10.1145/2775054.2694381>
- [2] T. Allen, B. Cooper, and R. Ge, "Fine-grain quantitative analysis of demand paging in unified virtual memory," *ACM Trans. Archit. Code Optim.*, vol. 21, no. 1, Jan. 2024. [Online]. Available: <https://doi.org/10.1145/3632953>
- [3] T. Allen and R. Ge, "Demystifying gpu uvm cost with deep runtime and workload analysis," in *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2021, pp. 141–150.
- [4] —, "In-depth analyses of unified virtual memory system for gpu accelerated computing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: <https://doi.org/10.1145/3458817.3480855>
- [5] R. Ausavarungrun, J. Landgraf, V. Miller, S. Ghose, J. Gandhi, C. J. Rossbach, and O. Mutlu, "Mosaic: a gpu memory manager with application-transparent support for multiple page sizes," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-50 '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 136–150. [Online]. Available: <https://doi.org/10.1145/3123939.3123975>
- [6] P. B., "Suv-micro2024," Sep. 2024. [Online]. Available: <https://doi.org/10.5281/zenodo.13743206>
- [7] P. B., G. Cox, J. Vesely, and A. Basu, "Suv: Static analysis guided unified virtual memory," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 293–308.
- [8] C.-H. Chang, J. Han, A. Sivasubramaniam, V. Sharma Mailthody, Z. Qureshi, and W.-M. Hwu, "Gmt: Gpu orchestrated memory tiering for the big data era," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 464–478. [Online]. Available: <https://doi.org/10.1145/3620666.3651353>

- [9] C.-H. Chang, A. Kumar, and A. Sivasubramaniam, "To move or not to move? page migration for irregular applications in over-subscribed gpu memory systems with dynamap," in *Proceedings of the 14th ACM International Conference on Systems and Storage*, ser. SYSTOR '21. New York, NY, USA: Association for Computing Machinery, 2021.
- [10] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A benchmark suite for heterogeneous computing," in *2009 IEEE International Symposium on Workload Characterization (IISWC)*, 2009, pp. 44–54.
- [11] S. Chien, I. Peng, and S. Markidis, "Performance evaluation of advanced features in cuda unified memory," in *2019 IEEE/ACM Workshop on Memory Centric High Performance Computing (MCHPC)*, 2019, pp. 50–57.
- [12] S. Choi, T. Kim, J. Jeong, R. Ausavarungnirun, M. Jeon, Y. Kwon, and J. Ahn, "Memory harvesting in Multi-GPU systems with hierarchical unified virtual memory," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 625–638. [Online]. Available: <https://www.usenix.org/conference/atc22/presentation/choi-sangjin>
- [13] E. Choukse, M. B. Sullivan, M. O'Connor, M. Erez, J. Pool, D. Nellans, and S. W. Keckler, "Buddy compression: Enabling larger memory for deep learning and hpc workloads on gpus," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 926–939.
- [14] I. D. Dio Lavore, D. Maffi, M. Arnaboldi, A. Delamare, D. Bonetta, and M. D. Santambrogio, "Grout: Transparent scale-out to overcome uvm's oversubscription slowdowns," in *2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2024, pp. 696–705.
- [15] D. Ganguly, R. Melhem, and J. Yang, "An adaptive framework for oversubscription management in cpu-gpu unified memory," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2021, pp. 1212–1217.
- [16] D. Ganguly, Z. Zhang, J. Yang, and R. Melhem, "Interplay between hardware prefetcher and page eviction policy in cpu-gpu unified virtual memory," in *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*, 2019, pp. 224–235.
- [17] —, "Adaptive page migration for irregular data-intensive applications under gpu memory oversubscription," in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2020, pp. 451–461.
- [18] R. Garg, A. Mohan, M. Sullivan, and G. Cooperman, "Crum: Checkpoint-restart support for cuda's unified memory," in *2018 IEEE International Conference on Cluster Computing (CLUSTER)*, 2018, pp. 302–313.
- [19] R. Gayatri, K. Gott, and J. Deslippe, "Comparing managed memory and ats with and without prefetching on nvidia volta gpus," in *2019 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2019, pp. 41–46.
- [20] S. Go, H. Lee, J. Kim, J. Lee, M. K. Yoon, and W. W. Ro, "Early-adaptor: An adaptive framework for proactive uvm memory management," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2023, pp. 248–258.
- [21] S. Grauer-Gray, L. Xu, R. Searles, S. Ayalasomayajula, and J. Cavazos, "Auto-tuning a high-level language targeted to gpu codes," in *2012 Innovative Parallel Computing (InPar)*, 2012, pp. 1–10.
- [22] Y. Gu, W. Wu, Y. Li, and L. Chen, "Uvmbench: A comprehensive benchmark suite for researching unified virtual memory in gpus," *arXiv preprint arXiv:2007.09822*, 2020.
- [23] P. Harish and P. J. Narayanan, "Accelerating large graph algorithms on the gpu using cuda," in *Proceedings of the 14th International Conference on High Performance Computing*, ser. HiPC'07. Berlin, Heidelberg: Springer-Verlag, 2007, p. 197–208.
- [24] C.-C. Huang, G. Jin, and J. Li, "Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1341–1355. [Online]. Available: <https://doi.org/10.1145/3373376.3378530>
- [25] T. B. Jablin, J. A. Jablin, P. Prabhu, F. Liu, and D. I. August, "Dynamically managed data for cpu-gpu architectures," in *Proceedings of the Tenth International Symposium on Code Generation and Optimization*, ser. CGO '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 165–174. [Online]. Available: <https://doi.org/10.1145/2259016.2259038>
- [26] T. B. Jablin, P. Prabhu, J. A. Jablin, N. P. Johnson, S. R. Beard, and D. I. August, "Automatic cpu-gpu communication management and optimization," in *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '11. New York, NY, USA: Association for Computing Machinery, 2011, p. 142–151. [Online]. Available: <https://doi.org/10.1145/1993498.1993516>
- [27] Z. Jin and J. S. Vetter, "A benchmark suite for improving performance portability of the sycl programming model," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2023, pp. 325–327.
- [28] J. Jung, J. Kim, and J. Lee, "Deepum: Tensor migration and prefetching in unified memory," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 2, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 207–221. [Online]. Available: <https://doi.org/10.1145/3575693.3575736>
- [29] M. Khairy, V. Nikiforov, D. Nellans, and T. G. Rogers, "Locality-centric data and threadblock management for massive gpus," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 1022–1036.
- [30] H. Kim, J. Sim, P. Gera, R. Hadidi, and H. Kim, "Batch-aware unified memory management in gpus for irregular workloads," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1357–1370. [Online]. Available: <https://doi.org/10.1145/3373376.3378529>
- [31] H. Kim and H. Han, "GPU thread throttling for page-level thrashing reduction via static analysis," *The Journal of Supercomputing*, Dec. 2023. [Online]. Available: <https://link.springer.com/10.1007/s11227-023-05787-y>
- [32] C. Li, R. Ausavarungnirun, C. J. Rossbach, Y. Zhang, O. Mutlu, Y. Guo, and J. Yang, "A framework for memory oversubscription management in graphics processing units," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 49–63. [Online]. Available: <https://doi.org/10.1145/3297858.3304044>
- [33] L. Li and B. Chapman, "Compiler assisted hybrid implicit and explicit gpu memory management under unified address space," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: <https://doi.org/10.1145/3295500.3356141>
- [34] M. Lin, Y. Feng, G. Cox, and H. Jeon, "Forest: Access-aware gpu uvm management," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 137–152. [Online]. Available: <https://doi.org/10.1145/3695053.3731047>
- [35] M. Lin, K. Zhou, and P. Su, "Drpsum: Guiding memory optimization for gpu-accelerated applications," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 3, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 164–178. [Online]. Available: <https://doi.org/10.1145/3582016.3582044>
- [36] X. Long, X. Gong, B. Zhang, and H. Zhou, "An intelligent framework for oversubscription management in cpu-gpu unified memory," *J. Grid Comput.*, vol. 21, no. 1, Feb. 2023. [Online]. Available: <https://doi.org/10.1007/s10723-023-09646-1>
- [37] S. W. Min, V. S. Malthody, Z. Qureshi, J. Xiong, E. Ebrahimi, and W.-m. Hwu, "Emogi: efficient memory-access for out-of-memory graph-traversal in gpus," *Proc. VLDB Endow.*, vol. 14, no. 2, p. 114–127, Oct. 2020. [Online]. Available: <https://doi.org/10.14778/3425879.3425883>
- [38] A. Nihaal and M. Mutyam, "Selective memory compression for gpu memory oversubscription management," in *Proceedings of the 53rd International Conference on Parallel Processing*, ser. ICPP '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 189–198. [Online]. Available: <https://doi.org/10.1145/3673038.3673058>
- [39] NVIDIA, "Nvidia blackwell architecture whitepaper," <https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>, accessed: August 10, 2025.

- [40] NVIDIA, "NVIDIA UVM," <https://github.com/NVIDIA/open-gpu-kernel-modules/tree/main/kernel-open/nvidia-uvm>, accessed: August 10, 2025.
- [41] —, "OpenGPU-Kernel-Modules," <https://github.com/NVIDIA/open-gpu-kernel-modules>, accessed: August 10, 2025.
- [42] Z. Qureshi, V. S. Maitlody, I. Gelado, S. Min, A. Masood, J. Park, J. Xiong, C. J. Newburn, D. Vainbrand, I.-H. Chung, M. Garland, W. Dally, and W.-m. Hwu, "Gpu-initiated on-demand high-throughput storage access in the bam system architecture," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 325–339. [Online]. Available: <https://doi.org/10.1145/3575693.3575748>
- [43] J. Ren, J. Luo, K. Wu, M. Zhang, H. Jeon, and D. Li, "Sentinel: Efficient tensor migration and allocation on heterogeneous memory systems for deep learning," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 598–611.
- [44] M. Rhu, M. Sullivan, J. Leng, and M. Erez, "A locality-aware memory hierarchy for energy-efficient gpu architectures," in *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-46. New York, NY, USA: Association for Computing Machinery, 2013, p. 86–98. [Online]. Available: <https://doi.org/10.1145/2540708.2540717>
- [45] G. Schieffer, J. Wahlgren, J. Ren, J. Faj, and I. Peng, "Harnessing integrated cpu-gpu system memory for hpc: a first look into grace hopper," in *Proceedings of the 53rd International Conference on Parallel Processing*, ser. ICPP '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 199–209. [Online]. Available: <https://doi.org/10.1145/3673038.3673110>
- [46] C. Shao, J. Guo, P. Wang, J. Wang, C. Li, and M. Guo, "Oversubscribing gpu unified virtual memory: Implications and suggestions," in *Proceedings of the 2022 ACM/SPEC on International Conference on Performance Engineering*, ser. ICPE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 67–75. [Online]. Available: <https://doi.org/10.1145/3489525.3511691>
- [47] H. Shin, S. Bang, H. Park, and D. Kim, "Safe: Sharing-aware prefetching for efficient gpu memory management with unified virtual memory," *IEEE Computer Architecture Letters*, vol. 24, no. 1, pp. 117–120, 2025.
- [48] T. Sultana, B. Allen, and A. Qasem, "Intelligent data placement on discrete gpu nodes with unified memory," in *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques*, ser. PACT '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 139–151. [Online]. Available: <https://doi.org/10.1145/3410463.3414651>
- [49] J. R. Tramm and A. R. Siegel, "Memory bottlenecks and memory contention in multi-core monte carlo transport codes," *Annals of Nuclear Energy*, vol. 82, pp. 195–202, 2015, joint International Conference on Supercomputing in Nuclear Applications and Monte Carlo 2013, SNA + MC 2013. Pluri- and Trans-disciplinarity, Towards New Modeling and Numerical Simulation Paradigms. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306454914004332>
- [50] N. Vijaykumar, E. Ebrahimi, K. Hsieh, P. B. Gibbons, and O. Mutlu, "The locality descriptor: A holistic cross-layer abstraction to express data locality in gpus," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018, pp. 829–842.
- [51] L. Wang, M. Jahre, A. Adileho, and L. Eeckhout, "Mdm: The gpu memory divergence model," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 1009–1021.
- [52] Y. Wang, B. Li, M. T. I. Ziad, L. Eeckhout, J. Yang, A. Jaleel, and X. Tang, "Oasis: Object-aware page management for multi-gpu systems," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1678–1692.
- [53] T. T. Yeh, R. N. Green, and T. G. Rogers, "Dimensionality-aware redundant simt instruction elimination," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1327–1340. [Online]. Available: <https://doi.org/10.1145/3373376.3378520>
- [54] Q. Yu, B. Childers, L. Huang, C. Qian, H. Guo, and Z. Wang, "Coordinated page prefetch and eviction for memory oversubscription management in gpus," in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2020, pp. 472–482.
- [55] Y. Yu, S. Kang, and Y. Park, "A compiler-based approach for gpgpu performance calibration using tlp modulation (wip paper)," in *Proceedings of the 20th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*, ser. LCTES 2019. New York, NY, USA: Association for Computing Machinery, 2019, p. 193–197. [Online]. Available: <https://doi.org/10.1145/3316482.3326343>
- [56] H. Zhang, Y. Zhou, Y. Xue, Y. Liu, and J. Huang, "G10: Enabling an efficient unified gpu memory and storage architecture with smart tensor migrations," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 395–410. [Online]. Available: <https://doi.org/10.1145/3613424.3614309>
- [57] Z. Zhang, T. Allen, F. Yao, X. Gao, and R. Ge, "Tunnels for bootlegging: Fully reverse-engineering gpu tlbs for challenging isolation guarantees of nvidia mig," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 960–974. [Online]. Available: <https://doi.org/10.1145/3576915.3616672>
- [58] T. Zheng, D. Nellans, A. Zulfiqar, M. Stephenson, and S. W. Keckler, "Towards high performance paged memory for gpus," in *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2016, pp. 345–357.
- [59] W. Zhu, G. Cox, J. Vesely, M. Hairgrove, A. L. Cox, and S. Rixner, "Uvm discard: Eliminating redundant memory transfers for accelerators," in *2022 IEEE International Symposium on Workload Characterization (IISWC)*, 2022, pp. 27–38.